Your Name: Caleb Zhong

Your Andrew ID: yuzhong4

Homework 5

0 Introduction

0.1 Collaboration and Originality

1. Did you receive help of any kind from anyone (other than the instructor or TAs) in developing your software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone who provided help, and describe the type of help that you received. No

2. Did you give help of any kind to anyone in developing their software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone that you helped, and describe the type of help that you provided. No

3. Did you examine anyone else's software for this assignment (Yes or No)? Do not describe software provided by the instructor. No

4. Are you (or the course instructor) the author of every line of source code submitted for this assignment (Yes or No)? If you answered No: Yes
    a. identify the software that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

5. Are you the author of every word of your report (Yes or No)? If you answered No: Yes
    a. identify the text that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

1 Passage Selection

1.1 Parameters for Passage Selection

Fixed Parameters

- `agentDepth = 5`: The top 5 documents retrieved by the first-stage ranker were used by the RAG agent.
- `agent:rag:maxTitleLength = 15`: Up to 15 tokens from the document title were included in passage construction.
- `agent:rag:prompt = 1`: The default prompt (Prompt 1) was used for all configurations to ensure consistency across experiments.

Retrieval Methods and Parameters

1. BM25 Retrieval

BM25 rankings were obtained from the provided `.inRank` file to reduce computational cost.

- $k_1 = 1.2, b = 0.75$

The initial ranking depth was set to 1000 documents.

2. Dense Retrieval

The top-ranked documents were retrieved directly from the FAISS index, with a ranking depth of 1000 documents.

Passage Selection Parameters

For passage selection, two strategies were compared:

1. First passage selection (firstp):

- `agent:rag:psgCnt = 1`
- The first passage of each document was used directly without additional scoring.

2. Best passage selection (bestp):

- `agent:rag:psgCnt = 6`
- The most relevant passage was selected from the first 6 passages of each document using dense vector similarity

Passage Construction Parameters

We varied the passage length to analyze its impact: `agent:rag:psgLen ∈ {50, 100, 150, 200}`

The passage stride was defined as: `agent:rag:psgStride = psgLen - 10`

This ensures overlapping passages to preserve contextual continuity.

1.2 Experimental Results for Passage Selection

Metrics	BM25 retrieval, first, 5 passages Varying Passage Length			
	50	**100**	**150**	**200**
	Exp-1.1a	**Exp-1.1b**	**Exp-1.1c**	**Exp-1.1d**
exact	32.50	35.00	35.00	35.00
f1	49.64	51.78	52.92	52.89
MRR	0.6643	0.6643	0.6643	0.6643
P@1	0.6.000	0.6000	0.6000	0.6000
P@5	0.4150	0.4150	0.4150	0.4150
time	01:21	01:18	01:30	01:05

Metrics	BM25 retrieval, bestp, 5 passages Varying Passage Length			
	50	**100**	**150**	**200**
	Exp-1.2a	**Exp-1.2b**	**Exp-1.2c**	**Exp-1.2d**
exact	30.00	32.50	40.00	37.50
f1	49.55	52.90	54.17	49.50
MRR	0.6643	0.6643	0.6643	0.6643
P@1	0.6000	0.6000	0.6000	0.6000
P@5	0.4150	0.4150	0.4150	0.4150
time	02:10	03:26	03:55	03:37

Metrics	dense retrieval, firstp, 5 passages Varying Passage Length			
	50	**100**	**150**	**200**
	Exp-1.3a	**Exp-1.3b**	**Exp-1.3c**	**Exp-1.3d**
exact	17.50	22.50	17.50	20.00
f1	28.69	35.50	33.19	36.63
MRR	0.4544	0.4544	0.4544	0.4544
P@1	0.3500	0.3500	0.3500	0.3500
P@5	0.2100	0.2100	0.2100	0.2100
time	01:05	02:23	02:51	04:51

Metrics	dense retrieval, bestp, 5 passages Varying Passage Length			
	50	**100**	**150**	**200**
	Exp-1.4a	**Exp-1.4b**	**Exp-1.4c**	**Exp-1.4d**
exact	17.50	32.50	20.00	22.50
f1	33.31	45.62	26.61	37.10
MRR	0.4544	0.4544	0.4544	0.4544
P@1	0.3500	0.3500	0.3500	0.3500
P@5	0.2100	0.2100	0.2100	0.2100
time	02:42	02:58	05:03	05:21

1.3 Discussion for Passage Selection

From the experimental results, we can see that the quality of the initial retrieval heavily dictates the success of the downstream RAG generation. BM25 consistently outperforms the dense ranker in both initial ranking metrics (`MRR` 0.6643 vs. 0.4544) and final generation quality. However, this gap is likely exacerbated by the evaluation design itself. The automatically generated relevance judgments label a document as relevant only if it contains the exact answer string, natively favoring lexical keyword-matching algorithms like BM25. Furthermore, could this performance drop also be tied to the 300-token truncation limit in our FAISS index? If the specific entity needed to answer a question is buried deep within a long Wikipedia article, the dense document vector simply fails to encode it, leaving the retriever completely blind to the actual answer and severely capping its maximum recall.

This initial retrieval bottleneck directly explains the suppressed `exact` and `f1` generation scores observed in the dense pipeline. Because of the potential 300-token cutoff and the dense model's inherent tendency to favor broad topic similarity over specific fact extraction, the RAG agent is frequently fed passages that are topically relevant but lack the necessary hard facts. Even if the LLM successfully synthesizes a semantically correct answer based on that context, it fails to output the exact required entity word results in a score of zero under the strict Exact and F1 evaluation scripts.

Building on the initial retrieval, the passage selection strategy also exposes a critical trade-off between positional heuristics and localized semantic density. The `firstp` strategy provides stable but artificially capped results because it relies on the flawed assumption that crucial information is always front-loaded in a document. Conversely, `bestp` actively searches for the most relevant text span, cutting out introductory filler to feed the LLM a highly concentrated context window. By providing this focused semantic signal, `bestp` consistently achieves the highest overall scores, such as pushing the BM25 exact match score to 40.00, proving that minimizing the noise-to-signal ratio fundamentally raises the accuracy ceiling of the LLM.

The relationship between passage length and accuracy further illustrates the limits of LLM context processing. For BM25 under `bestp`, performance peaks at 150 tokens but drops noticeably at 200 tokens. At 50 tokens, the context is too fragmented for the model to confidently extract an answer, but pushing the window to 200 tokens leads to some irrelevant sentences, causing the LLM to suffer from a dispersed attention. This sensitivity is even sharper for the dense ranker, where Exact match peaks at 100 tokens before dropping at 150, likely because the wider window forces the embedding to represent a broader, mixed topic that dilutes the specific answer entity. Ultimately, while `bestp` at carefully tuned lengths maximizes accuracy, the time metrics show it frequently doubles or triples execution time due to exhaustive sliding-window scoring.

2 Prompt Engineering

2.1 Custom Prompts

Prompt 2

```
[
{
"role": "system",
"content": "You are a question answering system. Answer the question using
only the information provided in the context. Provide a short answer only."
},
{
"role": "user",
"content": f"Question: {question}\nContext: {' '.join(passages)}\nAnswer:"
}
]
```

Hypothesis:

This prompt explicitly enforces grounding by restricting the model to use only the provided passages. Constraining the model to rely strictly on retrieved context will reduce hallucination and improve answer precision, especially when retrieval quality is high.

Prompt 3

```
[
{
"role": "system",
"content": "Answer the question with a very short phrase (one or two words if
possible). Do not include explanations."
},
{
"role": "user",
"content": f"Question: {question}\nContext: {' '.join(passages)}\nAnswer:"
}
]
```

Hypothesis:

This prompt enforces extreme brevity in responses. Encouraging shorter answers may better align with evaluation metrics such as exact match, which favor concise responses, but may reduce recall if the model omits necessary details.

Prompt 4

```
[
{
"role": "system",
"content": "The context may contain irrelevant or misleading information. Identify the most relevant part and use it to answer the question. Provide a short answer without explanation."
},
{
"role": "user",
"content": f"Question: {question}\nContext: {' '.join(passages)}\nAnswer:"
}
]
```

Hypothesis:

This prompt explicitly acknowledges that retrieved passages may be noisy. By guiding the model to filter irrelevant information, this prompt may improve robustness when retrieval quality is imperfect.

Prompt 5

```
[
{
"role": "system",
"content": "Find the exact answer from the context and copy it directly. Do not rephrase or explain."
},
{
"role": "user",
"content": f"Question: {question}\nContext: {' '.join(passages)}\nAnswer:"
}
]
```

Hypothesis:

This prompt biases the model toward extractive behavior. Encouraging the model to copy answers directly from the context will improve exact match, as it reduces paraphrasing, but may hurt performance if the answer is not explicitly present as a span.

Prompt 6

```
[
{
"role": "system",
"content": "First think about the answer based on the context, then provide the final answer in a short phrase. Only output the final answer."
},
```

```
{
"role": "user",
"content": f"Question: {question}\nContext: {' '.join(passages)}\nAnswer:"
}
]
```

Hypothesis:

This prompt introduces implicit chain-of-thought reasoning, while still constraining the final output. Encouraging intermediate reasoning may help the model better utilize context for complex questions, improving `f1`, even though the reasoning itself is not output.

2.2 Parameters for Prompt Engineering

Fixed Parameters

- `agentDepth = 5`: The top 5 documents retrieved by the first-stage ranker were used by the RAG agent.
- `agent:rag:maxTitleLength = 15`: Up to 15 tokens from the document title were included in passage construction.
- `agent:rag:prompt = 1`: The default prompt (Prompt 1) was used for all configurations to ensure consistency across experiments.

Retrieval Methods and Parameters

1. BM25 Retrieval

BM25 rankings were obtained from the provided `.inRank` file to reduce computational cost.

- $k_1 = 1.2, b = 0.75$

The initial ranking depth was set to 1000 documents.

2. Dense Retrieval

Dense rankings were reused from a dense `.inRank` file generated from the FAISS-based dense retriever.

Passage Selection Settings

For this experiment, prompt effects were isolated by fixing the passage-selection strategy to the best-performing configuration for each retrieval method from Experiment 1.

1. BM25 Condition

- `Passage selection strategy: bestp`
- `agent:rag:psgCnt = 6`
- `agent:rag:psgLen = 150`
- `agent:rag:psgStride = 140`

2. Dense Condition

- `Passage selection strategy: bestp`
- `agent:rag:psgCnt = 6`
- `agent:rag:psgLen = 100`
- `agent:rag:psgStride = 190`

2.3 Experimental Results for Prompt Engineering

	BM25 retrieval					
	Prompt 1 Exp-2.1a	**Prompt 2 Exp-2.1b**	**Prompt 3 Exp-2.1c**	**Prompt 4 Exp-2.1d**	**Prompt 5 Exp-2.1e**	**Prompt 6 Exp-2.1f**
exact	40.00	15.00	40.00	10.00	37.50	35.00
f1	54.17	35.87	51.46	36.17	45.85	48.71
MRR	0.6643	0.6643	0.6643	0.6643	0.6643	0.6643
P@1	0.6000	0.6000	0.6000	0.6000	0.6000	0.6000
P@5	0.4150	0.4150	0.4150	0.4150	0.4150	0.4150
Time	05:00	08:35	05:05	05:15	06:00	05:26

	dense retrieval					
	Prompt 1 Exp-2.2a	**Prompt 2 Exp-2.2b**	**Prompt 3 Exp-2.2c**	**Prompt 4 Exp-2.2d**	**Prompt 5 Exp-2.2e**	**Prompt 6 Exp-2.2f**
exact	32.50	10.00	25.00	22.50	22.50	17.50
f1	44.85	22.21	32.75	30.91	31.14	32.01
MRR	0.4544	0.4544	0.4544	0.4544	0.4544	0.4544
P@1	0.3500	0.3500	0.3500	0.3500	0.3500	0.3500
P@5	0.2100	0.2100	0.2100	0.2100	0.2100	0.2100
time	02:59	06:42	04:29	03:42	03:11	05:02

2.4 Discussion for Prompt Engineering

From the result we can see that The baseline Prompt 1 consistently achieves the best performance. This suggests that its design strikes an effective balance between grounding and flexibility. In particular, the role definition ("You are a helpful chatbot") may improve task alignment by clearly specifying the model's behavior, while the instruction to provide a short answer without explanation avoids unnecessary verbosity. Therefore, I think Prompt 1 works well may because it guides the model without over-constraining it.

However, the hypothesis for Prompt 2 is clearly not supported, as it performs the worst under both BM25 and dense retrieval. The main cause may be that the constraint is too restrictive. Many questions require light inference, normalization, or partial evidence aggregation, but this prompt forces the model to only answer when the answer is explicitly stated. As a result, the model abstains too often, leading to a sharp drop in both exact match and F1.

For Prompt 3 with short answer constraint, it performs best among the custom prompts and is close to the baseline, partially supporting the hypothesis. The improvement can be attributed to better alignment with the evaluation metric: concise answers reduce formatting variation and improve exact match.

For Prompt 4, which is the most interesting, the results differ significantly between retrieval methods. While it performs poorly under BM25, it achieves relatively better performance under dense retrieval. This difference may be explained by the nature of retrieved evidence. BM25 relies on lexical matching, which often retrieves documents that contain query terms but are not semantically focused on the answer. As a result, the context may contain multiple partially relevant or misleading passages. When Prompt 4 instructs the model to identify and filter noise, the model faces a harder selection problem and may become overly cautious, leading to degraded performance.

In contrast, dense retrieval tends to produce semantically more coherent and topically consistent passages, even if they are not always exact matches. In this case, the "noise-aware" instruction becomes beneficial: the model can more reliably identify the most relevant passage within a relatively clean context, improving answer extraction.

Prompt 5 performs well, especially under BM25. This supports the hypothesis that encouraging the model to copy answers directly from the context improves exact match by reducing paraphrasing. Since BM25 tends to retrieve passages with higher lexical overlap, answers are more likely to appear explicitly, making extractive behavior effective. However, its advantage is less clear under dense retrieval, suggesting that extractive prompting is most effective when answers appear as explicit spans in the retrieved text.

Last for Prompt 6, which also achieves competitive performance. This suggests that allowing the model to internally reason can help in some cases, but does not provide a consistent advantage. Since most questions in this setting are factoid-based, performance is more dependent on retrieval quality than reasoning complexity. Therefore, reasoning-oriented prompts are not harmful but offer limited gains when the task primarily requires fact extraction.

3 Custom Experiments

3.1 Design of the Custom Experiments

Baseline Configuration: BM25 → RAG (bestp)

- `agentDepth = 5, agentDepth = 5`
- `passage selection = bestp`
- `psgCnt = 6, psgLen = 150`
- `psgStride = 140, maxTitleLength = 15`
- `prompt = Prompt 1`

The baseline system follows the best-performing configuration identified in previous experiments. It is designed to provide a strong reference point, as it represents the optimal combination of retrieval, passage selection, and prompt design identified in Experiment 1 and Experiment 2.

Custom Configuration 1: BM25 → RAG (firstp)

This configuration replaces the best passage selection strategy with a simpler heuristic but only the first passage of each document is used, while all other parameters remain the same as the baseline.

- `passage selection = firstp`
- `psgCnt = 1, psgLen = 150`

The motivation is to test whether the expensive passage scoring process in `bestp` is necessary. Many documents, especially in structured corpora such as Wikipedia, tend to present key information near the beginning. Therefore, a strong positional prior may already capture sufficient evidence for answering questions.

The hypothesis is that `firstp` may achieve comparable performance to `bestp` with significantly lower computational cost, suggesting that complex passage selection may not always be required in RAG pipelines.

Custom Configuration 2: BM25 → BERT-6L → RAG

BERT reranking:

- model = `ms-marco-MiniLM-L-6-v2`
- `rerankDepth = 100, psgLen = 100, psgStride = 75, psgCnt = 6`
- `scoreAggregation = maxp`
- `maxTitleLength = 16`

RAG's configuration is same as baseline

This configuration introduces a semantic reranking stage before RAG. BM25 is used to retrieve the top 1000 documents, and a 6-layer BERT cross-encoder reranks the top 100 documents before passing them to the RAG module.

The motivation is based on prior findings that BERT rerankers are highly effective at improving top-ranked precision by capturing semantic relevance beyond lexical matching.

The hypothesis is that semantic reranking can improve the quality of the top documents fed into the RAG module, leading to higher answer accuracy, particularly for queries involving vocabulary mismatch or paraphrasing.

Custom Configuration 3: BM25 → BERT-6L → RAG (Prompt 3)

This configuration builds on Configuration 3 by modifying the prompt used in the RAG stage. Instead of the baseline prompt, Prompt 3 is used, which enforces a very short answer format (e.g., one or two words).

The motivation is to investigate the interaction between retrieval quality and answer generation constraints. While Prompt 3 alone performs slightly worse than the baseline prompt, it may better align with exact-match evaluation when the input evidence is already highly accurate.

The hypothesis is that combining high-quality semantic reranking with a concise answer constraint can further improve exact match performance, by reducing variability in generated answers.

Custom Configuration 4: BM25 → RAG (psgCnt = 10)

This configuration increases the number of passages extracted per document from 6 to 10 (psgCnt = 10), while reducing passage length to 100 tokens to control noise. All other settings remain identical to the baseline.

- `passage selection = bestp`
- `psgCnt = 10, psgLen = 100`

The motivation is to explore whether improving context coverage at the generation stage can compensate for limitations in retrieval. At first, I supposed to use PRF to improve retrieval recall and provide richer evidence, however, due to the change in the dataset, PRF is no longer applicable because it requires access to inverted index statistics. Therefore, we replace PRF with a generation-side alternative by increasing passage coverage.

The hypothesis is that increasing the number of passages can improve answer accuracy by providing more complete evidence, but may also introduce noise that negatively affects generation.

Custom Configuration 5: Dense → BERT-6L → RAG

This configuration introduces a semantic reranking stage on top of dense retrieval. While dense retrieval captures semantic similarity, prior results suggest that it often lacks the precision required for accurate answer extraction, especially under strict exact-match evaluation.

The motivation is to investigate whether a BERT reranker can refine the candidate set produced by dense retrieval and recover relevant documents that are semantically similar but poorly ranked. This design isolates whether the weakness of dense retrieval lies in its ranking quality rather than its ability to retrieve relevant content.

The hypothesis is that semantic reranking can significantly improve the effectiveness of dense retrieval by reordering the candidate set, leading to better alignment between retrieved evidence and the requirements of the RAG generation stage.

Custom Configuration 6: Dense → RankFusion → RAG

In this configuration, I designed a custom `rankfusion` module that combines two independent initial rankings (BM25 and dense retrieval) into a single candidate set before the RAG stage. The fusion is implemented as a lightweight reranking step that merges the primary and secondary rankings using interleaving strategy, and replaces the original ranking with the fused result. (**RankFusion.py**)

Rank Fusion reranking:

- type = rankfusion
- rerankDepth = 1000
- rankfusion:secondaryRankPath = Dense (.inRank file)
- rankfusion:method = interleave
- rankfusion:maxInputRank = 1000
- rankfusion:maxOutputRank = 100

The motivation is to explore whether combining lexical and semantic retrieval signals can produce a more informative candidate set than either method alone. BM25 is known for strong lexical precision, while dense retrieval captures semantic similarity but may lack exact matching accuracy. Instead of applying sequential reranking, this design explicitly integrates both signals at the retrieval level.

The hypothesis is that rank fusion can improve RAG performance by increasing the diversity and coverage of relevant evidence, allowing the downstream generation model to access complementary information that would not be available from a single retrieval method.

3.2 Experimental Results for Custom Experiments

Metrics	Baseline (n/a)	Custom 1 Exp-3.1b	Custom 2 Exp-3.1c	Custom 3 Exp-3.1d	Custom 4 Exp-3.1e	Custom 5 Exp-3.1f	Custom 6 Exp-3.1g
exact	40.00	35.00	40.00	45.00	42.50	22.50	40.00
f1	54.17	52.92	54.36	51.86	59.31	35.81	50.52
MRR	0.6643	0.6643	0.7017	0.7017	0.6643	0.6340	0.6659
P@1	0.6000	0.6000	0.6000	0.6000	0.6000	0.6000	0.6000
P@5	0.4150	0.4150	0.4300	0.4300	0.4150	0.2700	0.3500
time	03:55	01:57	09:18	10:27	03:11	09:09	02:52

3.3 Discussion of the Custom Experiments

From the result we can see that except Custom Configuration 5 (Dense), other custom configurations all achieved great performances. The results reveal several important insights about how retrieval quality, context construction, and model interaction affect RAG performance.

First, Custom 1 replaces the best passage selection with a simple positional heuristic, which leads to a drop in performance. It is reasonable since that although the first passage often contains useful information, the results confirm that relevance is not consistently front-loaded, and the `bestp` strategy is necessary to identify high-quality evidence spans. This suggests that passage selection remains a critical component even when retrieval quality is strong. However, despite the slight decrease in exact match, this configuration significantly reduces runtime, making it a favorable trade-off between efficiency and effectiveness.

In Custom 2, introducing semantic reranking improves retrieval metrics such as MRR and P@5, which is consistent with our hypothesis that BERT can better capture semantic relevance and refine the ranking of candidate documents. This confirms that the semantic reranking stage is effective at improving the quality of the ranked list.

However, this improvement does not translate into better answer generation performance, as the exact match and F1 scores remain similar to the baseline. This highlights a key observation that improving document ranking may not necessarily improve downstream generation, because RAG only utilizes a small subset of top-ranked documents. If the relevant evidence is already present within this subset, further refinement of the ranking provides limited benefit.

However, when semantic reranking is combined with a short-answer constraint (RAG with prompt 3), exact match improves to the highest value (45.00). This indicates a strong interaction effect that high-quality retrieval must be paired with appropriate output constraints to fully benefit evaluation metrics. It can be explained by better alignment between retrieval and evaluation. The BERT reranker increases the likelihood that the correct answer appears explicitly in the top passages, while Prompt 3 encourages the model to extract a concise answer span rather than generate a longer or paraphrased response. This reduces output variability and improves exact match.

Additionally, the short-answer constraint helps the model focus on the most relevant information when multiple passages are provided, effectively reducing noise during generation. The improvement suggests that better evidence alone is insufficient, how the model is guided to produce answers is equally important.

When we move to Custom 4 which replaces PRF with context expansion, it yields the best overall F1 score (59.31), supporting the hypothesis that generation-side evidence expansion can effectively compensate for retrieval limitations. Instead of modifying the query, increasing passage coverage also allows the model to access more relevant information within already retrieved documents and make a improvement.

In contrast, the dense-based pipeline with BERT reranking (Custom 5) performs significantly worse. Even with semantic reranking, dense retrieval fails to provide sufficiently precise evidence. This indicates

that the limitation of dense retrieval lies not only in ranking, but also in insufficient lexical grounding, which is critical under exact-match evaluation.

Finally, the hybrid rank fusion approach (Custom 6) does not provide a clear improvement over the baseline. Although combining BM25 and dense retrieval increases the diversity of candidate documents, this benefit does not translate into better answer quality.

A key reason is that RAG only uses a small subset of top-ranked documents (`agentDepth = 5`). If the fused ranking does not introduce higher-quality documents into this top portion, the additional candidates have little impact on generation. In practice, BM25 already provides strong top-ranked precision, so interleaving dense results may introduce lower-quality or less precise documents into the top ranks, effectively adding noise. As a result, the fusion strategy increases diversity but lacks a mechanism to prioritize truly relevant evidence, which limits its effectiveness in improving RAG performance.